Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — The three pieces of a graph
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The three pieces of a graph

A LangGraph graph has three pieces (Doc09):

- **state** — one `TypedDict` listing every field the graph carries;
- **nodes** — plain functions: take the state, return a dict of *only* the fields they change;
- **edges** — which node runs after which, from `START` to `END`.

For this step: two nodes, `clean_task` (strip the question) and `answer_node` (ask the model), joined in a straight line. `graph.invoke(starting_state)` runs them and returns the final state.

Put each piece in its own file, the way Doc09's Build Task did:

- `state.py` — `DocState` with `task`, `answer`, and `path: Annotated[list[str], operator.add]`. The `operator.add` tells LangGraph to **append** what a node returns for `path`, instead of replacing it — so `return {"path": ["clean_task"]}` adds one entry. Add `start_state(task)`, which returns every field with a starting value.
- `nodes.py` — each node returns its own name in `path`. `answer_node` uses a Doc05 chain: `ChatPromptTemplate.from_template(load_prompt("general")) | get_model() | StrOutputParser()`.
- `models/llm.py` — `get_model()` takes no arguments: nodes only receive the state, so they build the model themselves. `ChatOpenAI` reads the key that `load_config()` loaded.

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
build_graph():
    builder = StateGraph(DocState)
    builder.add_node("clean_task", clean_task)
    builder.add_node("answer_node", answer_node)
    builder.add_edge(START, "clean_task")
    builder.add_edge("clean_task", "answer_node")
    builder.add_edge("answer_node", END)
    return builder.compile()
```

`main.py`:

```
load_config()  (exit with one line on MissingConfigError)
graph = build_graph()
loop:
    task = input("You: ").strip()   — "/quit" stops, "" skips
    result = graph.invoke(start_state(task))
    print("path:", " -> ".join(result["path"]))
    print("Answer:", result["answer"])
```

Before running it, write down what the state looks like after each node. Then run it and compare with the printed `path`.

Then the first tests. Every case follows one pattern — start with `result = "OK"`, set `"WRONG"` if the check fails, and print `case N (what) -> {result}`:

- `tests/unit/test_config.py` — `require_env()` with a made-up setting name: missing and empty both raise `MissingConfigError`; `load_config()` loads once `OPENAI_API_KEY` is set.
- `tests/unit/test_nodes.py` — `clean_task({"task": " hi "})` gives `"hi"` and `["clean_task"]`; `start_state()` starts with an empty `path` and `answer`.
- `tests/integration/test_live_graph.py` — a `run(graph, thread_id, task)` helper, and case 1: a general question reaches `answer_node` and the answer mentions Paris.

Full solution: Show me the solution